

目录

文件包含与路径遍历漏洞修复报告 1

 目录 2

 1. 漏洞概述 2

 2. 漏洞一：路径遍历读取任意文件 2

 2.1 问题描述 2

 2.2 问题代码 2

 2.3 攻击示例 3

 2.4 实际攻击过程 3

 2.5 危害等级 3

 3. 漏洞二：无文件后缀限制 3

 3.1 问题描述 3

 3.2 可读取的文件类型 4

 4. 漏洞三：无错误处理导致信息泄露 4

 4.1 问题描述 4

 4.2 攻击方式 4

 5. 路径遍历漏洞原理 4

 5.1 什么是路径遍历 4

 5.2 攻击流程 4

 5.3 为什么会发生 5

 6. 修复方案 5

 6.1 修复对照表 5

 6.2 修复后核心代码 5

 6.3 修复原理 6

 7. 修复前后代码对比 6

 漏洞版（无任何校验） 6

 修复版（三层校验） 7

 8. 安全建议 7

 8.1 文件读取安全规范 7

 8.2 防御金字塔 8

 8.3 开发者检查清单 8

 8.4 推荐替代方案 8

 附录：OWASP 路径遍历防御指南 9

文件包含与路径遍历漏洞修复报告

项目名称：用户信息管理系统
报告日期：2026-07-13
审计目标：Flask 用户管理平台中的文件包含/路径遍历漏洞分析与修复
报告人：安全审计组

目录

1. 漏洞概述
2. 漏洞一：路径遍历读取任意文件
3. 漏洞二：无文件后缀限制
4. 漏洞三：无错误处理导致信息泄露
5. 路径遍历漏洞原理
6. 修复方案
7. 修复前后代码对比
8. 安全建议

1. 漏洞概述

在本次安全审计中，发现目标系统的动态页面加载功能存在 **3 项安全漏洞**，全部与文件路径处理相关。

编号	漏洞名称	风险等级	攻击入口
PT-001	路径遍历读取任意文件	严重	name 参数中的 ../
PT-002	无文件后缀限制	高危	可读取 .py、.db 等敏感文件
PT-003	路径不存在时信息泄露	中危	错误信息可被用于目录枚举

2. 漏洞一：路径遍历读取任意文件

2.1 问题描述

服务器直接使用用户输入的 name 参数拼接文件路径，没有对 ../ 等路径穿越字符做任何过滤或校验。

2.2 问题代码

```
@app.route("/page")
def page():
    name = request.args.get("name", "")

    # 直接拼接用户输入到路径中 —— 存在路径遍历!
    page_path = os.path.join("pages", name)

    if os.path.exists(page_path):
        with open(page_path, "r", encoding="utf-8") as f:
            page_content = f.read()
```

2.3 攻击示例

URL	读取的文件	后果
/page?name=../app.py	源代码	获取完整的应用源码
/page?name=../data/SQLite.db	SQLite 数据库文件	获取所有用户数据（含密码）
/page?name=../../etc/passwd	系统密码文件	获取系统用户列表
/page?name=../../etc/profile	环境变量	获取密钥等敏感配置

攻击者只需在浏览器地址栏修改 name 参数：
http://192.168.131.129:5000/page?name=../../etc/passwd
即可读取服务器上任意文件！

2.4 实际攻击过程

攻击者构造 URL：
/page?name=../app.py

服务器拼接路径：
os.path.join("pages", "../app.py") → "pages/../app.py" → "app.py"

服务器读取文件内容并返回给攻击者！

2.5 危害等级

攻击类型	可能性	影响
源代码泄露	极高	获取完整的应用逻辑和硬编码密钥
数据库泄露	极高	获取所有用户密码明文
系统配置文件泄露	高	获取数据库密码、API Key
服务器沦陷	中	结合其他漏洞进一步提权

3. 漏洞二：无文件后缀限制

3.1 问题描述

路由没有限制读取文件的后缀名，攻击者可以读取任意类型的文件。

3.2 可读取的文件类型

文件类型	示例路径	敏感程度
Python 源码	../app.py	极高
数据库	../data/users.db	极高
配置文件	../config.py	极高
环境变量	../../../../proc/self/environ	高
密码文件	../../../../etc/shadow	极高

4. 漏洞三：无错误处理导致信息泄露

4.1 问题描述

当请求的文件不存在时，虽然不会返回完整内容，但结合路径遍历可以逐级确认目录结构。

4.2 攻击方式

请求 /page?name=../app.py → 返回内容（文件存在）
请求 /page?name=../config.py → 返回"页面不存在"（文件不存在）

攻击者可以通过这种方式探测服务器上是否存在特定文件。

5. 路径遍历漏洞原理

5.1 什么是路径遍历

路径遍历（Path Traversal）是一种通过操纵文件路径中的 ../（上级目录）序列来访问本不应被访问的目录或文件的安全漏洞。

5.2 攻击流程

用户输入： ../../etc/passwd
↓
路径拼接： pages ../../etc/passwd
↓
实际路径： /etc/passwd（../ 让路径跳出了 pages/ 目录）
↓
文件读取： 返回 /etc/passwd 的内容

5.3 为什么会发生

原因	说明
信任用户输入	认为用户只会输入合法的页面名称
缺乏路径校验	没有检查最终路径是否在允许的范围内
直接拼接路径	使用 <code>os.path.join</code> 直接拼接用户输入
未规范路径	没有使用 <code>os.path.realpath</code> 解析 <code>../</code>

6. 修复方案

6.1 修复对照表

漏洞编号	修复措施	风险等级变化
PT-001	使用 <code>os.path.realpath</code> 规范化路径，校验是否在 <code>pages</code> 目录内	严重 → 已修复
PT-002	只允许 <code>.html</code> 后缀的文件	高危 → 已修复
PT-003	统一返回“页面不存在”，不区分文件是否存在	中危 → 已修复

6.2 修复后核心代码

```
import os

@app.route("/page")
def page():
    name = request.args.get("name", "")

    if not name:
        return render_template("index.html", page_error=" 未指定页面名称")

    # 【已修复】只允许 .html 后缀
    if not name.endswith(".html"):
        name = name + ".html"

    # 【已修复】使用 os.path.realpath 规范化路径，解析掉 ../ 等序列
    pages_dir = os.path.realpath("pages")
    page_path = os.path.realpath(os.path.join("pages", name))

    # 【已修复】检查路径是否在 pages 目录内，防止 ../ 逃逸
```

```

if not page_path.startswith(pages_dir):
    return render_template("index.html", page_error=" 页面不存在")

# 检查文件是否存在
if not os.path.exists(page_path):
    return render_template("index.html", page_error=" 页面不存在")

# 安全读取文件
with open(page_path, "r", encoding="utf-8") as f:
    page_content = f.read()

```

6.3 修复原理

```

# 攻击者输入: ../app.py
page_path = os.path.realpath(os.path.join("pages", "../app.py"))
# 实际结果: /workspace/user-management/app.py ← 真实的绝对路径

pages_dir = os.path.realpath("pages")
# 实际结果: /workspace/user-management/pages/ ← 允许的目录

# 校验: app.py 不在 pages/ 目录下
page_path.startswith(pages_dir) → False ← 拦截成功!

```

7. 修复前后代码对比

漏洞版（无任何校验）

```

@app.route("/page")
def page():
    name = request.args.get("name", "")

    if not name:
        return render_template("index.html", page_error=" 未指定页面名称")

    # 直接拼接用户输入，无校验
    page_path = os.path.join("pages", name)

    # 尝试加 .html 后缀
    if not os.path.exists(page_path):
        page_path = os.path.join("pages", name + ".html")

```

```

if os.path.exists(page_path):
    with open(page_path, "r", encoding="utf-8") as f:
        page_content = f.read()
    return render_template("index.html", page_content=page_content)
else:
    return render_template("index.html", page_error=" 页面不存在")

```

修复版（三层校验）

```

@app.route("/page")
def page():
    name = request.args.get("name", "")

    if not name:
        return render_template("index.html", page_error=" 未指定页面名称")

    # 第一层：限制后缀
    if not name.endswith(".html"):
        name = name + ".html"

    # 第二层：规范化路径
    pages_dir = os.path.realpath("pages")
    page_path = os.path.realpath(os.path.join("pages", name))

    # 第三层：校验路径前缀
    if not page_path.startswith(pages_dir):
        return render_template("index.html", page_error=" 页面不存在")

    if not os.path.exists(page_path):
        return render_template("index.html", page_error=" 页面不存在")

    with open(page_path, "r", encoding="utf-8") as f:
        page_content = f.read()
    return render_template("index.html", page_content=page_content)

```

8. 安全建议

8.1 文件读取安全规范

检查项	说明	优先级
① 路径规范化	使用 <code>os.path.realpath</code> 解析 <code>../</code>	必须
② 路径前缀校验	确认最终路径在允许的目录内	必须
③ 后缀白名单	只允许读取特定后缀的文件（如 <code>.html</code> ）	推荐
④ 使用文件 ID 代替路径	用数字 ID 映射文件，不暴露真实路径	推荐

8.2 防御金字塔

路径规范化校验	← 核心防线，防 <code>../</code> 逃逸
白名单目录校验	← 确认文件在允许范围内
后缀白名单	← 限制可读取的文件类型
文件 ID 映射	← 不暴露真实文件路径

8.3 开发者检查清单

- ☐ 永不直接拼接用户输入到文件路径
- ☐ 使用 `os.path.realpath` 规范化路径
- ☐ 检查规范化后的路径是否在允许的基准目录内
- ☐ 限制可读取的文件后缀
- ☐ 使用文件 ID 或名称映射，而不是用户提供的原始路径
- ☐ 统一错误信息，不区分“文件存在”和“文件不存在”
- ☐ 不要读取 `.py`、`.db`、`.json` 等包含敏感信息的文件

8.4 推荐替代方案

```
# 推荐：使用白名单映射，完全不暴露文件路径
PAGE_MAP = {
    "help": "help.html",
    "about": "about.html",
    "faq": "faq.html",
}

@app.route("/page")
def page():
    name = request.args.get("name", "")
    filename = PAGE_MAP.get(name)  # 如果 name 不在映射中，返回 None
    if not filename:
```



```
return render_template("index.html", page_error=" 页面不存在")

page_path = os.path.join("pages", filename)
# 此时 page_path 是完全可控的，不需要额外校验
```

使用白名单映射后，用户输入的 name 参数不再直接参与路径拼接，从设计上杜绝了路径遍历攻击。

附录：OWASP 路径遍历防御指南

OWASP（开放式 Web 应用程序安全项目）关于文件路径安全的建议：

1. **不要直接使用用户输入构造文件路径**
2. **使用白名单映射**：预定义允许访问的文件列表
3. **路径规范化后再校验**：使用 `realpath()` 或 `canonicalize()` 解析路径中的 `..` 和符号链接
4. **校验路径前缀**：确认规范化后的路径以允许的基准目录开头
5. **最小权限**：应用使用低权限用户运行，限制可访问的文件范围
6. **Chroot 或容器隔离**：将应用限制在特定的文件系统范围内

相关 CVE 示例：- CVE-2021-41773: Apache HTTP Server 路径遍历漏洞 (`..%2f` 绕过) - CVE-2021-42013: Apache HTTP Server 路径遍历 (`..%2e` 双重编码绕过) - CVE-2018-11776: Apache Struts 2 远程代码执行 (路径遍历 + OGNL 注入)

报告编写：Claude Code AI Assistant

报告版本：v1.0

参考标准：OWASP Top 10 (2021) - A01:2021 Broken Access Control

测试环境：Flask 用户管理系统

免责声明：本报告仅用于安全学习和教学目的